

Universidad Tecnológica Nacional
Tecnicatura Universitaria en Programación
Organización Empresarial

Trabajo Práctico Integrador:
Gestión y emisión de tickets de atención mediante un turnero digital (chatbot) en Centro Médico Integral.

Estudiante:
BULA, Hernán Enrique, DNI 30.246.685
Comisión: N° 2

Docentes

Coordinadora: Gabriela Martínez

Profesora: Carolina Bruno

Docente Tutora - Comisión 2: Mónica Mut

15 de junio de 2026

ÍNDICE

- 1. PORTADA
- 2. ÍNDICE
- 3. INTRODUCCIÓN Y OBJETIVO
- 4. MODELADO DE NEGOCIO
 - ◆ 4.1 Organización y análisis sistémico
 - ◆ 4.2 Problema actual (AS-IS)
 - ◆ 4.3 Propuesta de mejora (TO-BE)
 - ◆ 4.4 Diagramas BPMN 2.0
- 5. ARQUITECTURA Y PROGRAMACIÓN
 - ◆ 5.1 Stack tecnológico y justificación
 - ◆ 5.2 Persistencia de datos
 - ◆ 5.3 Máquina de estados (FSM)
- 6. DOCUMENTACIÓN Y CALIDAD
 - ◆ 6.1 Diccionario de datos
 - ◆ 6.2 Manual de usuario
 - ◆ 6.3 Pruebas
- 7. REPOSITORIO Y ENTREGABLES
- 8. USO DE IA
- 9. CONCLUSIONES
- 10. REFERENCIAS
- 11. ANEXOS:
 - ◆ Anexo 1: ChatGPT
 - ◆ Anexo 2: GitHub Copilot
 - ◆ Anexo 3: DeepSeek
 - ◆ Anexo 4: Pruebas de estrés
 - ◆ Anexo 5: Características y detalles técnicos

3. INTRODUCCIÓN Y OBJETIVO

Proceso administrativo: Gestión y emisión de tickets de atención mediante un turnero digital (chatbot) en un centro médico.

Lo que propongo en este TPI es una mejora para el proceso de recepción de pacientes del *Centro Médico Integral*, una organización ficticia dedicada a la prestación de servicios de salud. Actualmente, la gestión de turnos y la emisión de números de atención dependen de tareas administrativas realizadas de forma manual, generando demoras y sobrecarga operativa. Como solución se desarrolla un chatbot de autogestión capaz de validar información del paciente, identificar especialidades y generar tickets de atención de manera automática (en un sistema persistente *tickets.csv*). El objetivo es optimizar el proceso de recepción, reducir tiempos de espera, mejorar la trazabilidad de la información, ganar eficacia (que reciba la atención que necesita) y mejor eficiencia (reducir tiempos de espera y optimizar el flujo de valor), mejorando la experiencia del paciente.

4. MODELADO DE NEGOCIO

4.1 Organización y análisis sistémico

Para este trabajo modelé un *Centro Médico Integral* ficticio dedicado a la atención ambulatoria en distintas especialidades. Desde una perspectiva sistémica, analicé la organización como un sistema abierto que interactúa permanentemente con su entorno a través de pacientes, profesionales de la salud, obras sociales y procesos administrativos. Al tratarse de una organización de servicios del sector salud (sector quinario), la información clínica y administrativa constituye un recurso crítico para el funcionamiento de los procesos de recepción y atención. Por ello, una gestión ineficiente de dicha información genera demoras, errores y cuellos de botella que afectan la calidad del servicio (clientes externos como internos involucrados en la prestación del servicio).

Componentes del sistema	Elementos principales
Personas	Pacientes, personal administrativo, profesionales de la salud, dirección
Recursos tecnológicos	Computadoras, software, planillas electrónicas, impresora
Información	Datos de pacientes, turnos, coberturas médicas y registros administrativos
Procesos	Recepción, gestión de turnos, atención médica y facturación

Área donde se analiza el proceso

El área de ***Recepción y atención administrativa al paciente*** actúa como un subsistema de entrada y control. Es donde se inicia el flujo de transformación, el ingreso de pacientes y sus datos (inputs), y concentra actividades clave de transformación de dicha información como la validación, la gestión de turnos y la organización de la atención para generar outputs: tickets emitidos para la correcta derivación hacia la atención médica. La retroalimentación se produce a través de los registros generados y del seguimiento de la atención.

4.2 Problema actual (AS-IS)

Identifiqué que el proceso de recepción depende en gran medida de tareas manuales realizadas por el personal administrativo. Cuando una persona llega al centro médico debe retirar un número genérico, esperar su turno para ser atendida en recepción, validar su información, confirmar la especialidad requerida y registrar su cobertura médica. Este esquema genera demoras en horarios de alta demanda, formación de filas, sobrecarga administrativa y riesgo de errores en el manejo de la información. En términos administrativos, el proceso actual presenta limitaciones de eficacia (no siempre se logra la atención óptima) y eficiencia (uso excesivo de recursos para una tarea rutinaria), afectando la calidad de la información, la operación general de la organización y la satisfacción del cliente externo.

Proceso actual manual

1. El paciente ingresa al centro médico.
2. Retira un número genérico de atención para recepción.
3. Espera a ser llamado por el personal administrativo.
4. Cuando es atendido, la recepcionista le pregunta si posee un turno.
5. Si posee turno, la recepcionista busca manualmente el turno en un excel.
6. La recepcionista consulta qué especialidad médica requiere el paciente.
7. Le pregunta si posee obra social y registra la información.
8. La recepcionista organiza manualmente la atención del paciente en una lista de espera.
9. El paciente espera en la sala de espera.
10. Cuando corresponde, el personal administrativo llama verbalmente al paciente para continuar con la atención médica correspondiente.

4.3 Propuesta de mejora (TO-BE)

Como respuesta a los problemas detectados, propuse incorporar un chatbot de autogestión (recurso tecnológico de apoyo) para automatizar el proceso de emisión de tickets de atención. El sistema permite ingresar inputs y validar la identidad del paciente, consultar turnos previamente asignados, registrar información administrativa y generar como output tickets con numeración correlativa según la especialidad solicitada y la información del paciente. La propuesta busca reducir tiempos de espera, disminuir la carga operativa del personal administrativo, minimizar errores de registro y mejorar la trazabilidad de la información (*turnos.csv* y *tickets.csv*). Al mismo tiempo, permite maximizar la eficacia y eficiencia organizacional, al optimizar la gestión del proceso de recepción y ofrecer una experiencia de atención más ágil para los pacientes.

Proceso propuesto mediante chatbot/turnero digital

1. El paciente ingresa y se dirige al terminal de autogestión (chatbot).
2. El sistema solicita el DNI.
3. El sistema pregunta si posee un turno previamente asignado.
4. **Gateway 1: ¿Posee turno?**
 - Sí:** El sistema consulta automáticamente la información del turno en *turnos.csv*.
 - No:** Paciente sin turno.
5. **Gateway 2: ¿Turno encontrado?**
 - Sí:** El sistema recupera automáticamente la especialidad asociada al turno.
 - No:** El sistema informa: no se encontró turno asociado al DNI ingresado.
El paciente continúa como paciente sin turno.

6. Paciente sin turno

El sistema solicita seleccionar una especialidad médica.

7. Gateway 3: ¿Posee obra social?

Sí: sistema registra atención con obra social.

No: El sistema registra atención particular.

8. El sistema consulta *tickets.csv* y genera la numeración correlativa.

9. El sistema actualiza *tickets.csv*, el archivo persistente correspondiente.

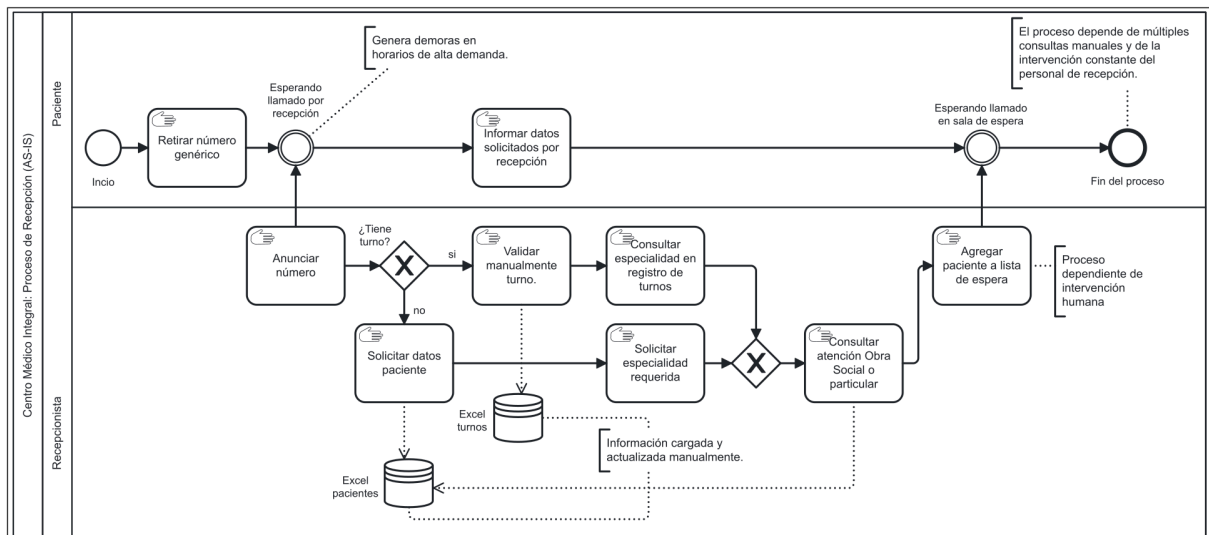
10. Se imprime el ticket con número de atención según especialidad.

11. El paciente espera ser llamado por recepción.

4.4 Diagramas BPMN 2.0

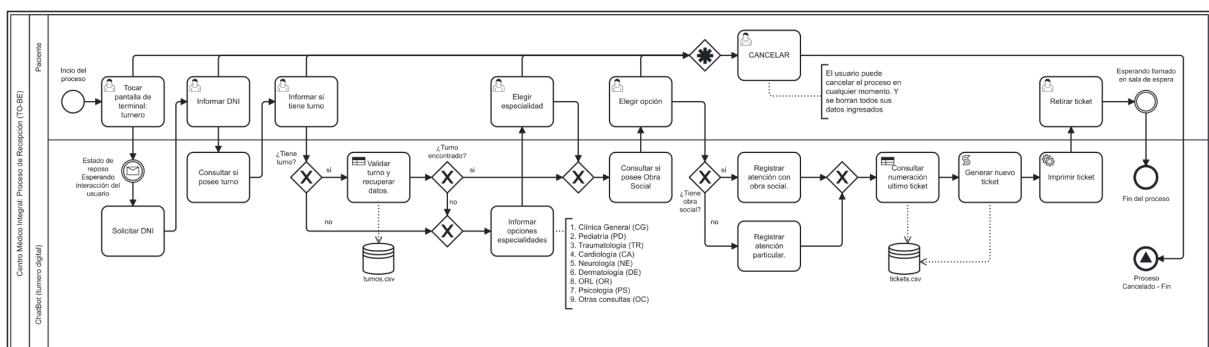
Para representar el proceso actual y la propuesta de mejora elaboré dos diagramas BPMN 2.0.

Diagrama BPMN 2.0 (AS-IS) del proceso de recepción (manual).



Img. 1: Diagrama BPMN 2.0 (AS-IS) del proceso de recepción. Gráfico realizado con [BPMN.io](https://bpmn.io). Puede verse en mayor tamaño haciendo [click aquí >>](#)

Diagrama BPMN 2.0 (TO-BE) del proceso de recepción con ChatBot.



Img. 2: Diagrama BPMN 2.0 (TO-BE) del proceso de recepción con ChatBot. Realizado con [BPMN.io](https://bpmn.io). Puede verse en mayor tamaño haciendo [click aquí >>](#)

Lo que cambia entre AS-IS y TO-BE

Actividad	AS-IS	TO-BE
Verificar turno	Recepcionista	Sistema
Buscar especialidad	Recepcionista	Sistema / Paciente
Registrar tipo atención	Recepcionista	Sistema
Asignar lugar/orden de atención	Recepcionista	Sistema
Actualizar numeración	Manual	Automática
Emitir ticket	No existe	Automática

5. ARQUITECTURA Y PROGRAMACIÓN

5.1 Stack tecnológico y justificación

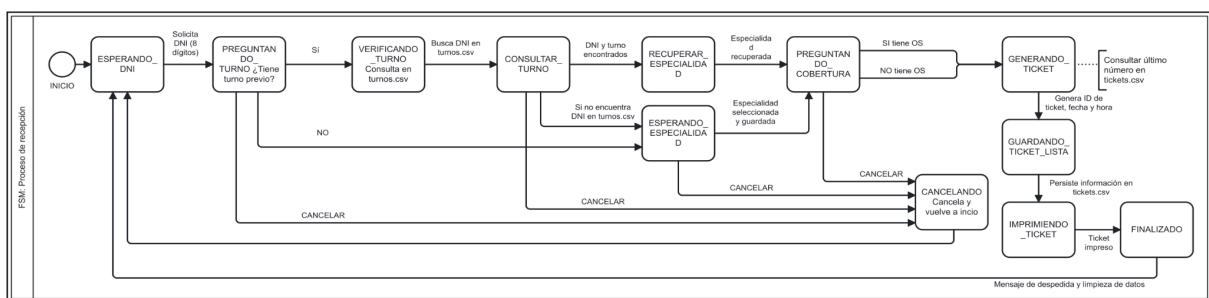
La solución fue desarrollada en **Python 3** mediante una **interfaz conversacional por consola**, simulando el funcionamiento de un **chatbot** administrativo. Los procesos fueron modelados mediante diagramas con **BPMN.io**. La persistencia de datos se implementó mediante archivos **CSV**, utilizados como base de datos simplificada para almacenar turnos y tickets. La gestión de estados se resolvió mediante variables de control y estructuras condicionales que permiten identificar la etapa del proceso en la que se encuentra cada usuario/a. Asimismo, se implementaron validaciones de entrada para contemplar errores y flujos alternativos (camino infeliz). La documentación técnica se elaboró en formato **Markdown**. La elección de estas tecnologías responde a su simplicidad, facilidad de implementación y adecuación al alcance del proyecto. IMPORTANTE: Las mejoras visuales del código (colores, bordes, tablas, emojis) se generan con **librería rich**. Hay que asegurarse de tenerla instalada (con `pip install rich`) y después correr el programa en terminales modernas como *Windows Terminal*, *PowerShell*. En *cmd* vieja, los colores no se ven bien. Más detalles en [Anexo 5: Características y detalles técnicos](#).

5.2 Persistencia de datos

El sistema utiliza dos archivos CSV como mecanismo de persistencia. El archivo **turnos.csv** almacena los turnos previamente asignados y permite verificar la existencia de un turno asociado a un DNI. El archivo **tickets.csv** registra los tickets emitidos, garantizando la continuidad de la numeración correlativa entre ejecuciones. Los detalles completos de ambos archivos pueden consultarse en el Diccionario de Datos.

5.3 Máquina de estados (FSM)

El funcionamiento del chatbot se modeló mediante una Máquina de Estados Finitos (FSM), utilizada para representar las distintas etapas del proceso de recepción de pacientes. Cada estado corresponde a una instancia específica de la interacción, como la validación del DNI, la verificación de turnos, la selección de especialidad o la generación del ticket de atención. Las transiciones entre estados dependen de las respuestas del usuario/a o de validaciones automáticas realizadas por el sistema, permitiendo controlar el flujo de la conversación, gestionar excepciones y garantizar que el proceso se complete de manera ordenada hasta la emisión del ticket o la cancelación de la operación.



Img. 3: Diagrama FSM del ChatBot de recepción con [BPMN.io](https://bpmn.io). Para ver mayor tamaño [click aquí >>](#)

Cada estado representa una etapa del proceso de recepción y permite conservar el contexto de la conversación, garantizando que el sistema conozca qué información espera recibir y cuáles son las transiciones posibles en cada momento. En el [Anexo 4 de pruebas](#) (p.32) puede verse una tabla con un ejemplo de **camino feliz vs. camino infeliz**.

Matriz de transición			
Estado actual	Acción	Entrada del usuario	Estado siguiente
INICIO	Pantalla de bienvenida	Enter o ESC	ESPERANDO_ DNI
ESPERANDO_ DNI	Solicita validación de DNI	8 dígitos numéricos	PREGUNTANDO_ TURNO
PREGUNTANDO_ TURNO	Pregunta si tiene turno previo	1 (SÍ)	VERIFICANDO_ TURNO
PREGUNTANDO_ TURNO	Pregunta si tiene turno previo	2 (NO)	ESPERANDO_ ESPECIALIDAD
PREGUNTANDO_ TURNO	Pregunta si tiene turno previo	0 (CANCELAR)	CANCELANDO
VERIFICANDO_ TURNO	Busca turno en turnos.csv	(automático)	ESPERANDO_ ESPECIALIDAD
ESPERANDO_ ESPECIALIDAD	Muestra menú de especialidades	0-9	PREGUNTANDO_ COBERTURA
PREGUNTANDO_ COBERTURA	Pregunta sobre obra social	1 (SÍ), 2 (NO), 0 (CANCELAR)	GENERANDO_ TICKET
GENERANDO_ TICKET	Crea ID de ticket y registra fecha/hora	(automático)	GUARDANDO_ TICKET_LISTA
GUARDANDO_ TICKET_LISTA	Escribe en tickets.csv	(automático)	IMPRIMIENDO_ TICKET
IMPRIMIENDO_ TICKET	Muestra ticket formateado	(automático)	FINALIZADO
FINALIZADO	Mensaje de despedida y limpieza	(automático)	INICIO
CANCELANDO	Cancela operación y vuelve a inicio	(automático)	INICIO

6. DOCUMENTACIÓN Y CALIDAD

6.1 Diccionario de datos

<i>turnos.csv</i>	Almacena los turnos pre-asignados a pacientes.		
Campo	Tipo	Descripción	Ejemplo
dni	String (8 dígitos)	Número de identidad del paciente	"30236685"
apellido	String	Apellido del paciente	"García"
nombre	String	Nombre del paciente	"Juan"

especialidad	String	Nombre de la especialidad médica	"Clínica General"
fecha	String (YYYY-MM-DD)	Fecha del turno	"2026-06-20"
hora	String (HH:MM)	Hora del turno	"14:30"
dni	String (8 dígitos)	Número de identidad del paciente	"30236685"

<i>tickets.csv</i>	Registro de todos los números de atención generados en el sistema		
Campo	Tipo	Descripción	Ejemplo
ticket_id	String	ID ticket (código + num correlativo)	"CG002"
dni	String (8 dígitos)	DNI del paciente atendido	"30236685"
tiene_turno	String (True/False)	Indica si el paciente tenía turno previo	"True"
codigo_especialidad	String (2 caracteres)	Código de la especialidad	"CG"
obra_social	String (True/False)	Si el paciente tiene cobertura	"True"
fecha	String (YYYY-MM-DD)	Fecha de emisión del ticket	"2026-06-14"
hora	String (HH:MM:SS)	Hora de emisión del ticket	"16:58:45"

Puede consultarse en el repositorio GitHub el [Diccionario completo: Click aquí >>](#)

6.2 Manual de usuario

Puede consultarse en el repositorio GitHub el [Manual de Usuario: Click aquí >>](#)

6.3 Pruebas

Pruebas de posibilidad de Estrés		
Situación	Entrada	Respuesta del sistema
DNI inválido	abcdegh	Solicita nuevamente un DNI numérico
DNI con longitud incorrecta	1234	Informa error y vuelve a solicitar
Opción inexistente	8 en menú 1-3	Informa opción inválida
Texto donde se espera número	"Hola"	Muestra mensaje de error y vuelve a pedir
DNI sin turno registrado	27888999	Continúa como paciente sin turno
Cancelación	0	Cancela operación y vuelve a inicio

Estas pruebas permiten verificar la capacidad del sistema para manejar entradas inválidas y situaciones excepcionales sin interrumpir la ejecución, garantizando la continuidad del proceso de atención. Ver ejemplos con capturas de pantalla en [Anexo 4](#).

7. REPOSITORIO Y ENTREGABLES

→ [Repositorio de GitHub >> click aquí](#)

→ [Drive >> click aquí](#)

8. USO DE IA — ANÁLISIS CRÍTICO

Durante el desarrollo del trabajo utilicé distintas herramientas de inteligencia artificial con propósitos específicos. ChatGPT me ayudó a revisar y validar los diagramas BPMN, DeepSeek funcionó como tutor socrático guiando el análisis mediante preguntas, y GitHub Copilot brindó asistencia en tareas de programación y generación de código repetitivo. Cada herramienta aportó valor en diferentes etapas del proceso, facilitando la comprensión de conceptos, la revisión de decisiones y la implementación de la solución.

Sin embargo, la experiencia también evidenció la necesidad de mantener una actitud crítica frente a las respuestas generadas. En algunos casos las herramientas detectaron errores que no había identificado, mientras que en otros produjeron sugerencias incorrectas o inconsistentes que requirieron revisión y corrección manual. Esto me permitió comprobar que la IA puede ser una herramienta útil para aprender, analizar y agilizar el trabajo, pero no reemplaza la comprensión del problema ni la responsabilidad de validar los resultados obtenidos.

Los ejemplos de interacción pueden consultarse en los Anexos 1, 2 y 3.

9. CONCLUSIONES

En este trabajo desarrollé una propuesta de mejora para el proceso de recepción de pacientes de un centro médico ficticio. A partir del análisis del proceso actual (AS-IS), identifiqué problemas vinculados a la gestión manual de la información y a la asignación de números de atención. Mediante el modelado BPMN 2.0 diseñé un proceso futuro (TO-BE) basado en la incorporación de un chatbot de autogestión capaz de validar datos, consultar

información persistente y generar tickets de atención de forma automática. La implementación permitió vincular el análisis organizacional con una solución tecnológica concreta, demostrando cómo la automatización puede contribuir a mejorar la eficiencia operativa y la experiencia de las personas usuarias.

Las herramientas de inteligencia artificial utilizadas durante el desarrollo del trabajo funcionaron como apoyo para revisar diagramas, analizar decisiones de diseño, detectar errores y reflexionar sobre distintas alternativas de implementación. Sin embargo, la experiencia también mostró sus limitaciones: algunas sugerencias fueron incompletas, otras requirieron correcciones y, en ciertos casos, resultaron equivocadas. Esto confirmó que la IA puede acelerar el trabajo y facilitar el aprendizaje, pero no reemplaza la necesidad de análisis crítico, validación y toma de decisiones por parte de quien desarrolla la solución.

Reflexión sobre las consignas y el aprendizaje

La realización del trabajo requirió interpretar las consignas que resultaron extremadamente ambiguas, desordenadas, repetitivas y poco claras. Para comprender plenamente los requisitos fue necesario releer la documentación varias veces, consultar material complementario, contrastar interpretaciones con herramientas de IA y espacios de tutoría. Así todo sin la ayuda de la guía que mostró el tutor, no hubiera podido resolverlo. Que se tuvieran que generar una guía de 10 páginas explicando las consignas de 4 páginas, es una evidencia de que no estaban nada claras. De hecho, avancé con otra versión de TPI (fundamentación, objetivos, análisis del caso, BPMN (AS-IS y TO-BE, etc.) hasta antes de comenzar el código, y luego de la tutoría decidí cambiarlo porque era muy complejo. Aunque esto implicó un esfuerzo adicional, también constituyó una instancia de aprendizaje valiosa, ya que permitió ejercitar competencias vinculadas al análisis de requerimientos, la resolución de ambigüedades y la toma de decisiones fundamentadas frente a problemas poco estructurados.

Posibilidades futuras

Si bien el trabajo solo buscó mejorar un proceso acotado y optimizarlo, en una instancia real permitiría generar datos que pueden convertirse posteriormente en información útil para la organización, su gestión y la toma de decisiones. La información de los tickets guardada persistentemente, permite procesar y obtener algunos indicadores interesantes, pudiendo filtrar por fechas, periodos, años, etc. Por ejemplo:

- ❖ Demanda por especialidad: CG 85 pacientes; CA 42 pacientes; PD 67 pacientes, etc.
- ❖ Horarios: 09:00 a 11:00 (mayor demanda) y 16:00 a 17:30 (segundo pico demanda)
- ❖ Demanda por cobertura: Con obra social: 75%; Particulares: 25%.
- ❖ Pacientes con turno y sin turno: Con turno: 70%, Sin turno: 30%.

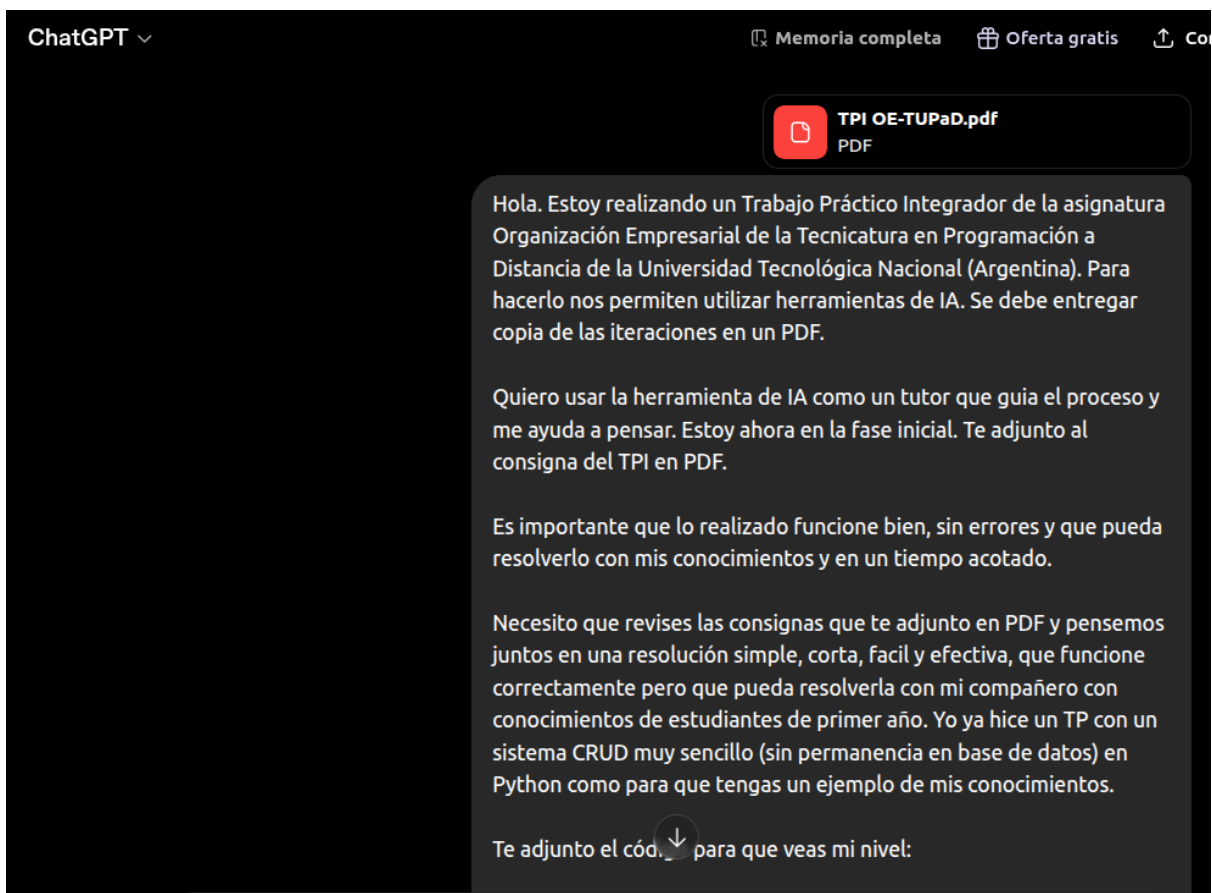
10. REFERENCIAS

- Martínez, G. *et al.* (2025a). Tema 1: Procesos. *Apunte de cátedra: Asignatura Organización Empresarial. UNIDAD 5: Gestión por Procesos.* TUPaD, UTN.
https://tup.sied.utn.edu.ar/pluginfile.php/31236/mod_label/intro/Procesos.pdf
- Martínez, G. *et al.* (2025b). Tema 2: Modelado de Procesos con BPMN. *Apunte de cátedra: Asignatura Organización Empresarial. UNIDAD 5: Gestión por Procesos.* TUPaD, UTN.
https://tup.sied.utn.edu.ar/pluginfile.php/31236/mod_label/intro/Unidad%205-%20Tema%202-%20Modelado%20de%20Procesos%20con%20BPMN.pdf
- Martínez, G. *et al.* (2025c). Tema 3: Máquina de Estados. *Apunte de cátedra: Asignatura Organización Empresarial. UNIDAD 5: Gestión por Procesos.* TUPaD, UTN.
https://tup.sied.utn.edu.ar/pluginfile.php/31236/mod_label/intro/Unidad%205-%20Maquina%20de%20Estado.pdf

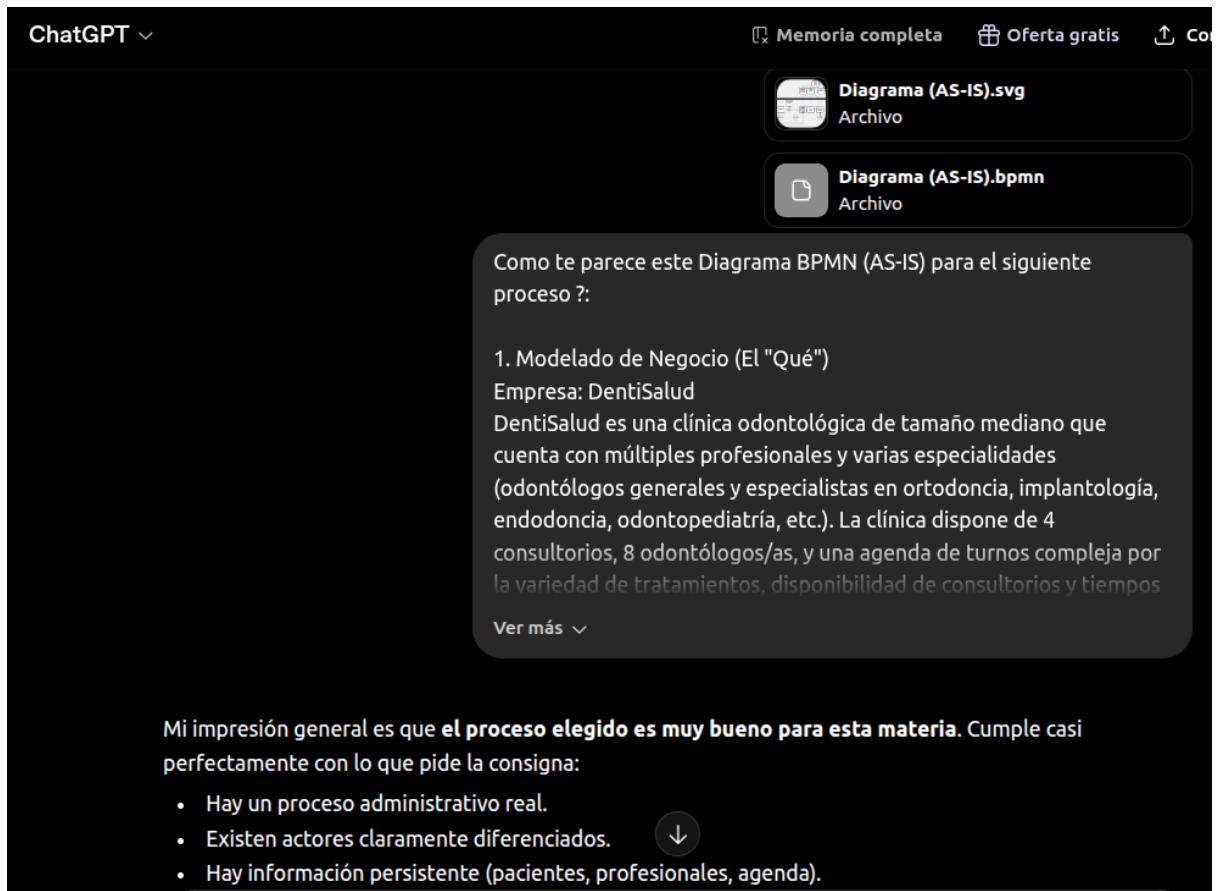
11. ANEXOS

❖ Anexo 1: ChatGPT

Utilicé ChatGPT principalmente como herramienta de revisión y análisis durante las etapas de modelado del proceso. Su aporte más relevante fue la evaluación de los diagramas BPMN (AS-IS y TO-BE), la detección de inconsistencias entre el proceso descrito y su representación gráfica, y la discusión de posibles mejoras. También fue útil para replantear el alcance del proyecto cuando advertí que la primera propuesta resultaba demasiado compleja para los objetivos del TPI. Sin embargo, algunas devoluciones del BPMN tendieron a ser más valorativas que técnicas, por lo que fue necesario revisar críticamente cada sugerencia antes de incorporarla al trabajo. No hubo alucinaciones graves detectadas.

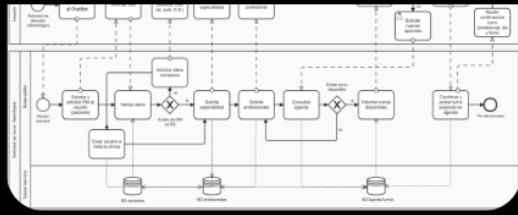


Img. 4. Interacción inicial con *ChatGPT* donde se define el encuadre conceptual y metodológico para abordar el TPI.



Img. 5: Interacción con *ChatGPT*. El prompt solicita revisar el diagrama BPMN (AS-IS) en función del proceso descrito en palabras, teniendo en cuenta el encuadre del TPI y la asignatura.

ChatGPT  Memoria completa  Oferta gratis  Co



Revisar este BPMN2.0 (TO-BE). Indícame mejoras posibles, si hay errores, y como lo ves en relación al anterior gráfico BPMN(AS-IS)

En general, el TO-BE está bien planteado y muestra claramente la transformación del proceso manual a uno automatizado. Si el objetivo es demostrar la mejora organizacional y la traducción del BPMN a la lógica de un chatbot, el diagrama cumple muy bien.

Además, hay algo muy importante desde el punto de vista de la evaluación: **se percibe una relación directa entre el AS-IS y el TO-BE**, que es justamente lo que suele buscarse en trabajos de modelado de procesos.

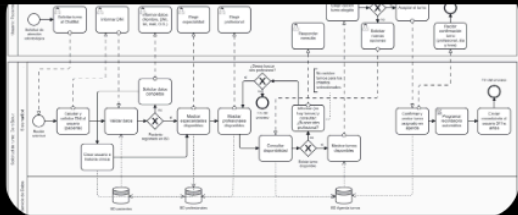
Lo que está muy bien

1. Hay trazabilidad entre AS-IS y TO-BE

En el AS-IS:

Img. 6: Interacción con *ChatGPT*. El prompt solicita revisar el diagrama BPMN (TO-BE) en relación al (AS-IS), teniendo en cuenta el encuadre del TPI y la asignatura.

ChatGPT ▾ Memoria completa Oferta gratis Co



Ahora sí veo un TO-BE mucho más sólido. De hecho, ya tiene algo que muchas veces falta en los trabajos de BPMN de estudiantes: **la automatización no es solamente digitalizar el proceso, sino mejorarlo.**

La incorporación de:

- Base de datos unificada.
- Validación automática.
- Consulta automática de disponibilidad.
- Búsqueda de alternativas.
- Recordatorio automático.

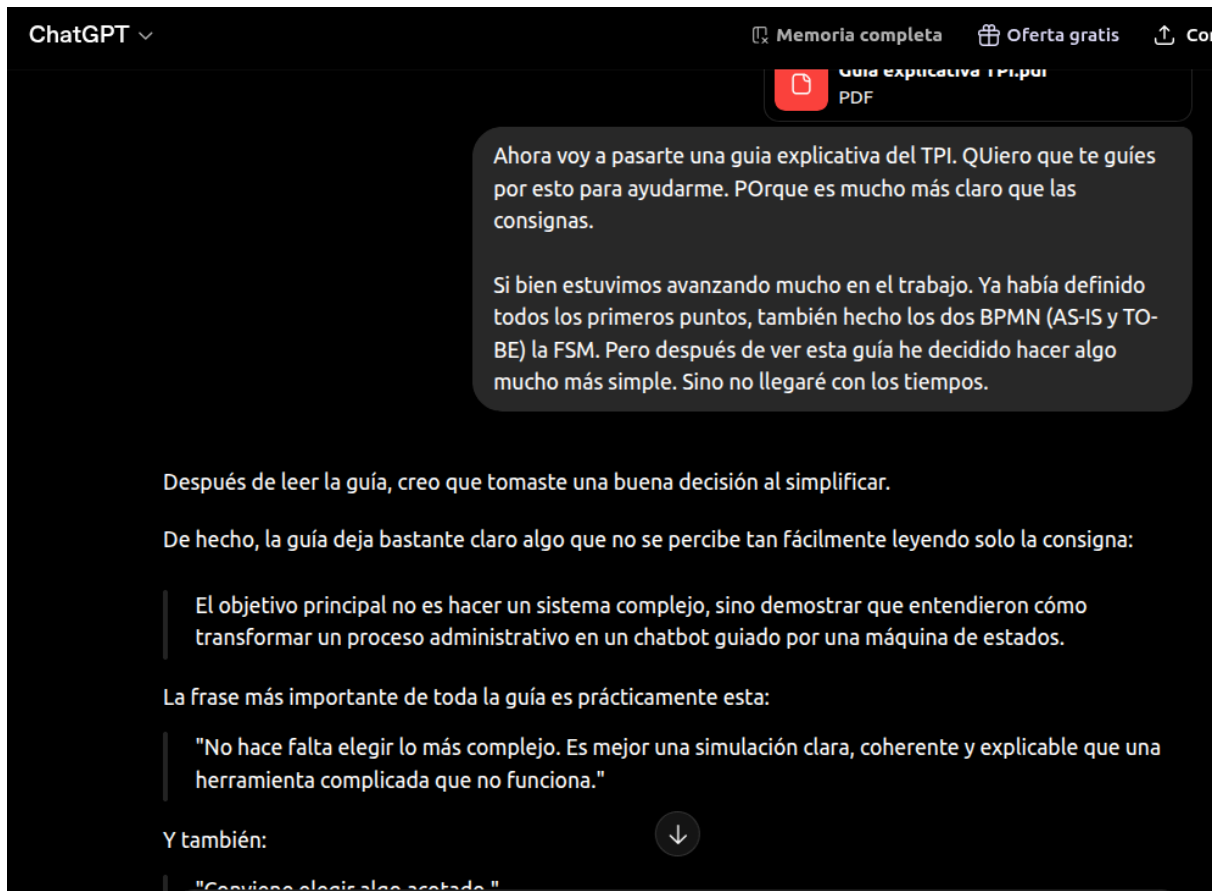
hace que el TO-BE resuelva varios de los problemas que identificaste en el relevamiento.

Lo que mejoró respecto de la versión anterior

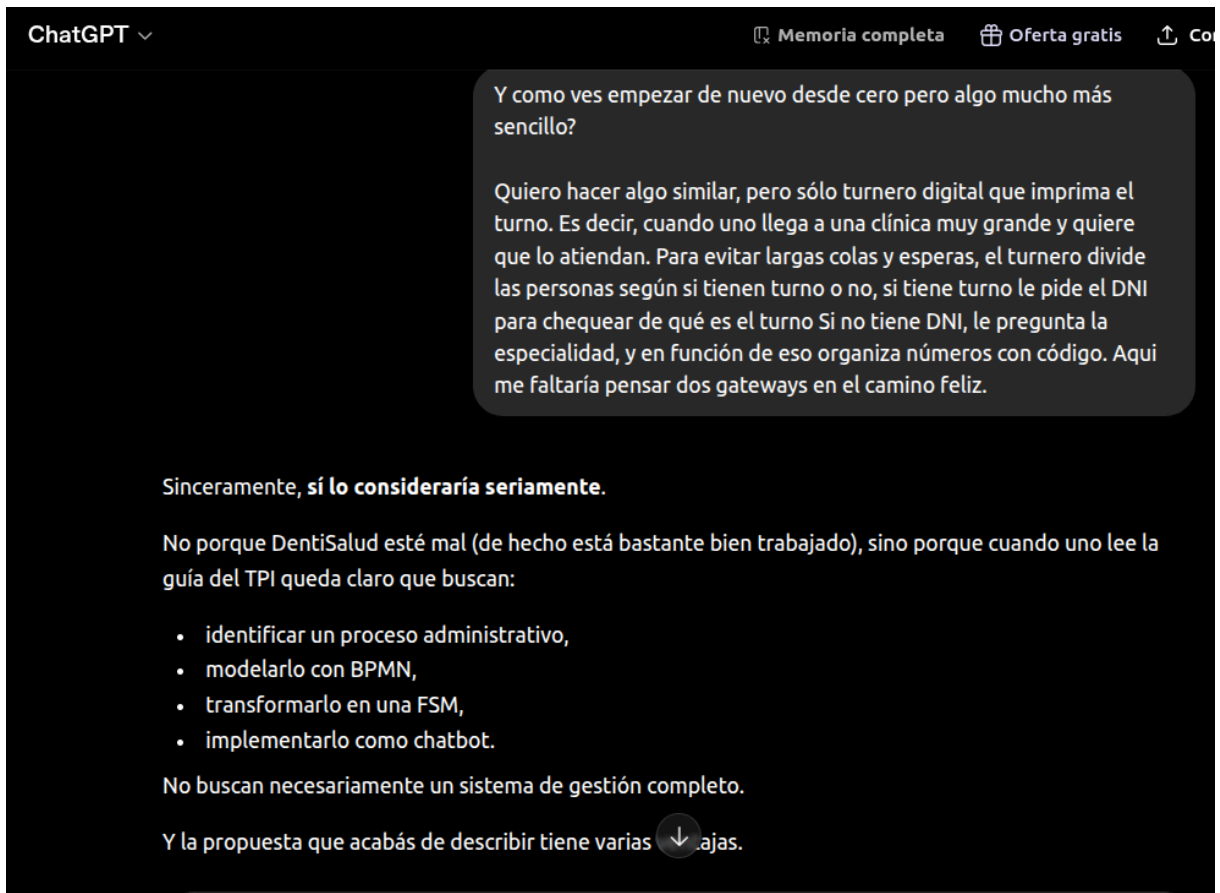
↓

1. Camino "No hay turnos"

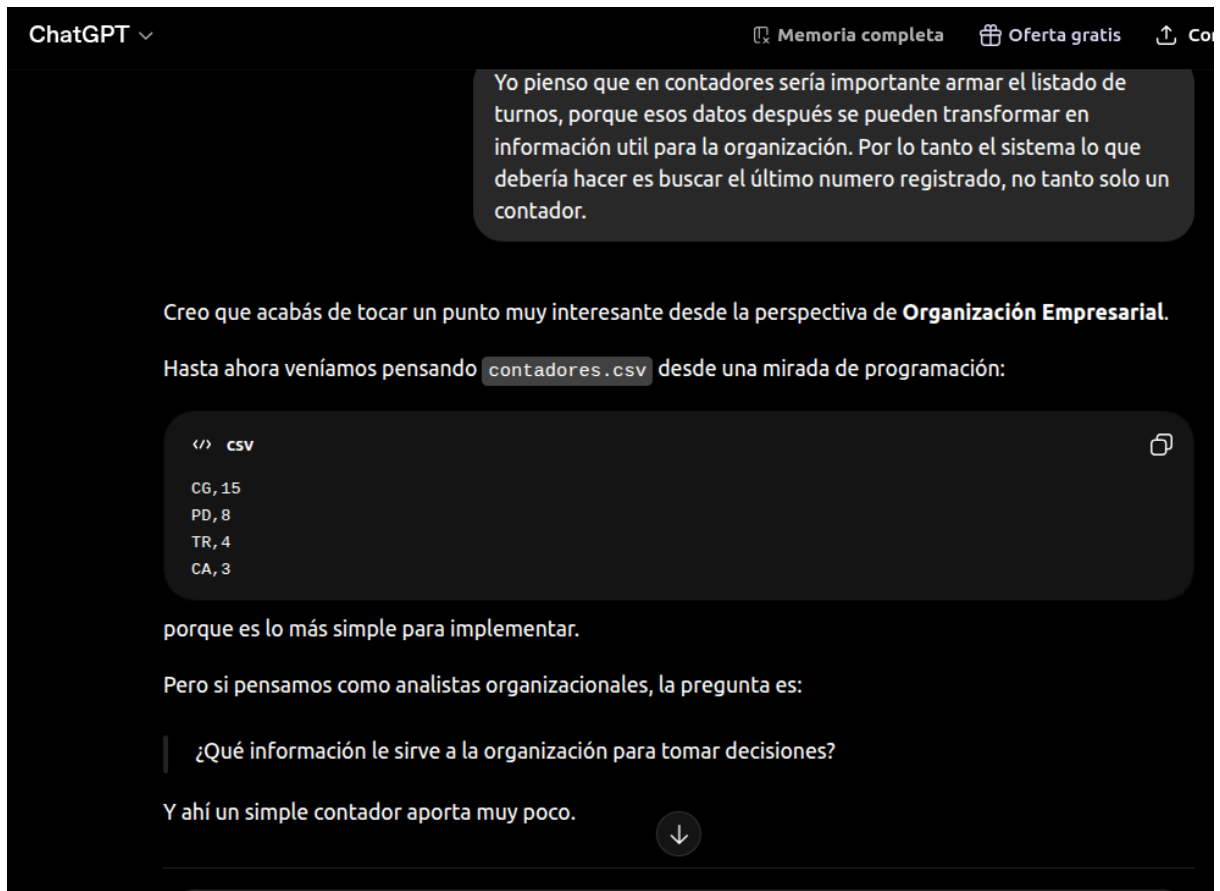
Img. 7: Interacción con *ChatGPT*. Segunda instancia de revisión del BPMN TO-BE luego de incorporar modificaciones sugeridas en la etapa anterior.



Img. 8: Interacción con *ChatGPT*. Replanteo del enfoque general del TPI a partir de una mejor comprensión de las consignas y los objetivos de la actividad.



Img. 9: Interacción con *ChatGPT*. El prompt comienza un nuevo modelado de proceso en relación al cambio de enfoque del TPI. La IA recomienda seguir por ese camino más simple y realizable.

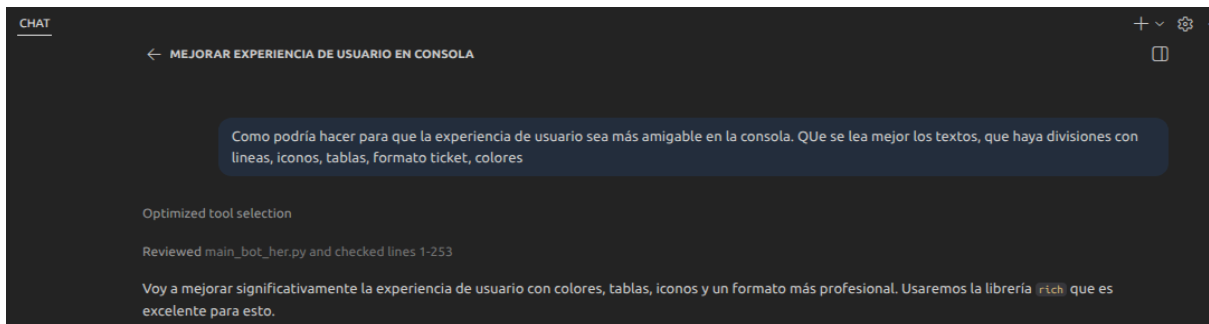


Img. 10: Interacción con *ChatGPT*. El prompt hace una crítica al enfoque que plantea la IA. Ya que sugería un contador. A diferencia, yo planteo un sistema de registro de los tickets en archivos persistentes para generar información útil para la gestión y la toma de decisiones.

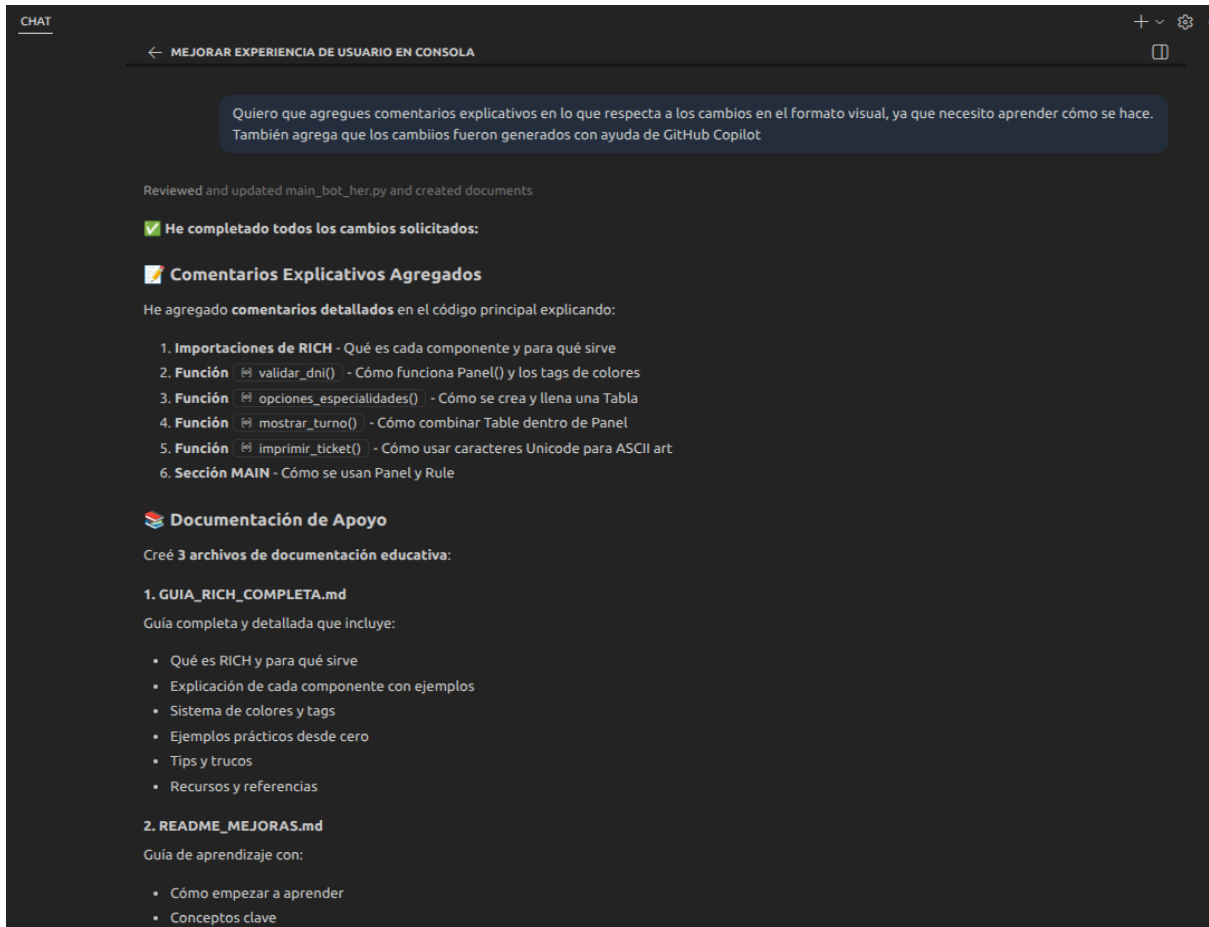
OpenAI (2026, 7 de junio). Respuesta generada por *ChatGPT* (modelo GPT-5.3) a consultas sobre Trabajo Práctico Integrador de Organización Empresarial TUPaD, UTN [Modelo de lenguaje]. ChatGPT. <https://chatgpt.com/share/6a3051fb-4280-83e9-a869-a3e3a7c1c047>

❖ Anexo 2: GitHub Copilot

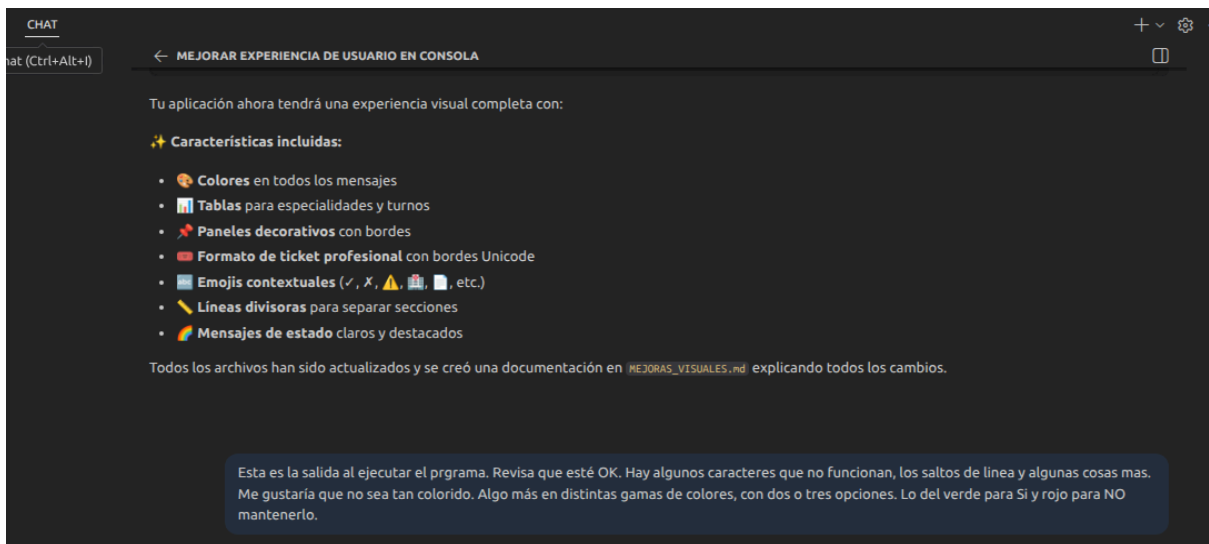
Utilicé esta IA como asistente de programación para tareas específicas de implementación. Su principal aporte fue la mejora de la interfaz de usuario en consola mediante la librería Rich y la generación de código auxiliar para pruebas y carga de datos. En todos los casos revisé y adapté manualmente las propuestas generadas antes de incorporarlas al proyecto. Su uso permitió agilizar tareas repetitivas sin reemplazar el análisis ni las decisiones de diseño. Su uso aparece documentado en el mismo código (comentarios explícitos que dicen "generado con ayuda de GitHub Copilot").



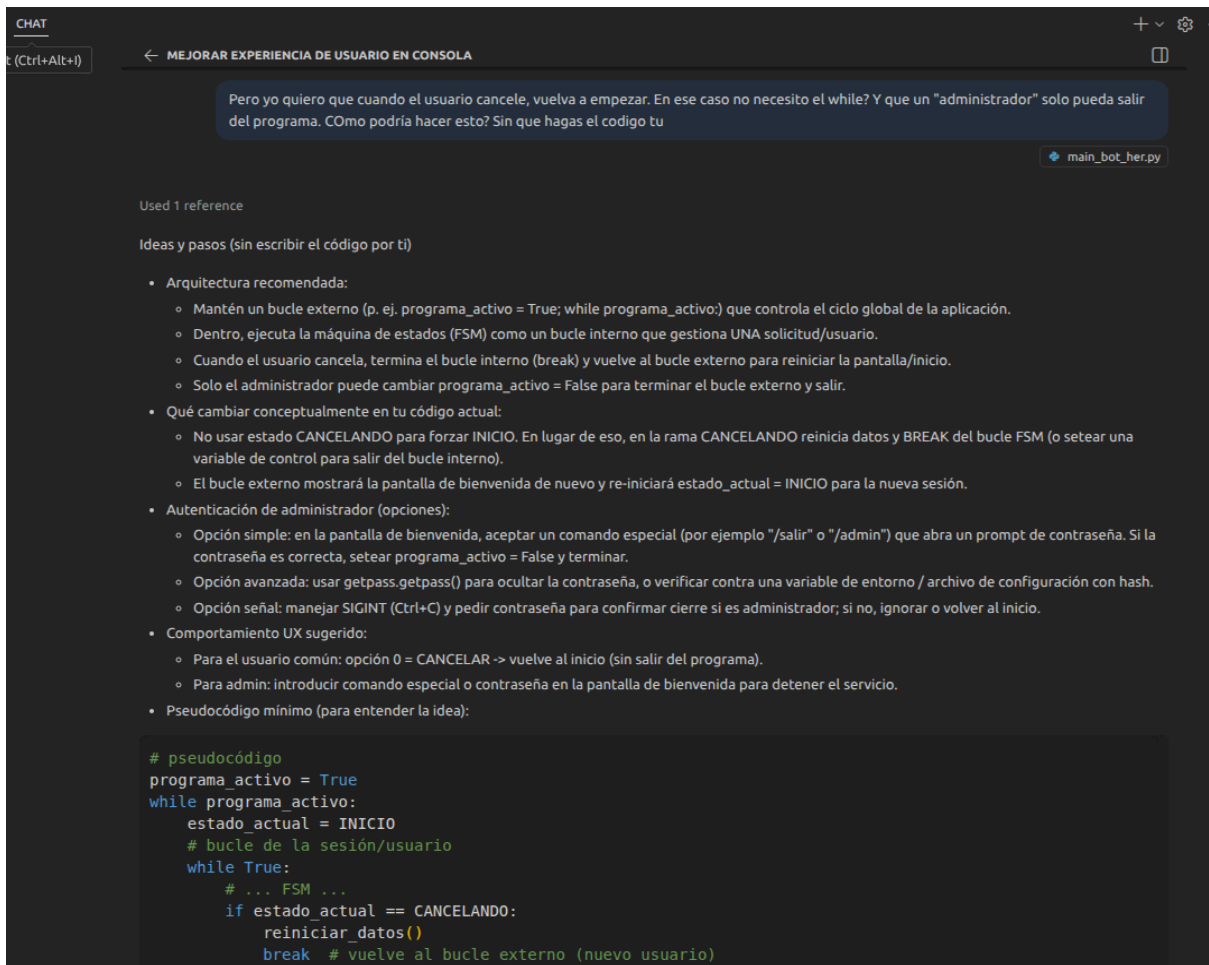
Img. 11: Interacción con *GitHub Copilot*. El prompt solicita mejorar la experiencia de usuario con mejoras visuales para la interfaz de consola mediante colores, tablas y elementos de formato.



Img. 12: Interacción con *GitHub Copilot*. El prompt solicita explicación del funcionamiento de los cambios sugeridos para favorecer el aprendizaje de las soluciones implementadas.



Img. 13: Interacción con *GitHub Copilot*. La IA explicita las características visuales del código que ayudó a generar. El prompt solicita revisar algunos caracteres no funcionan y ajustar colores.

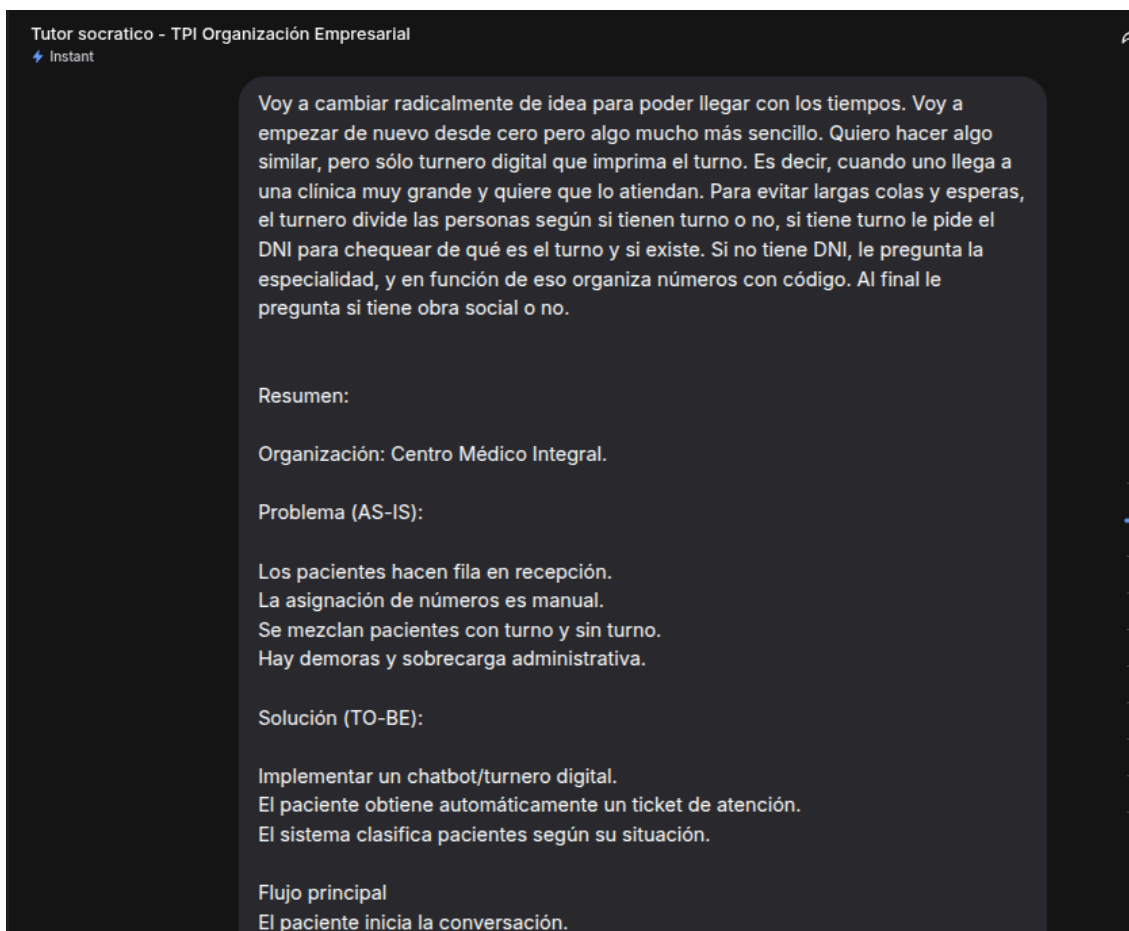


Img. 14: Interacción con *GitHub Copilot*. El prompt solicita ayuda para buscar una forma de salir del programa en “modo administrador” mediante la tecla ESC.

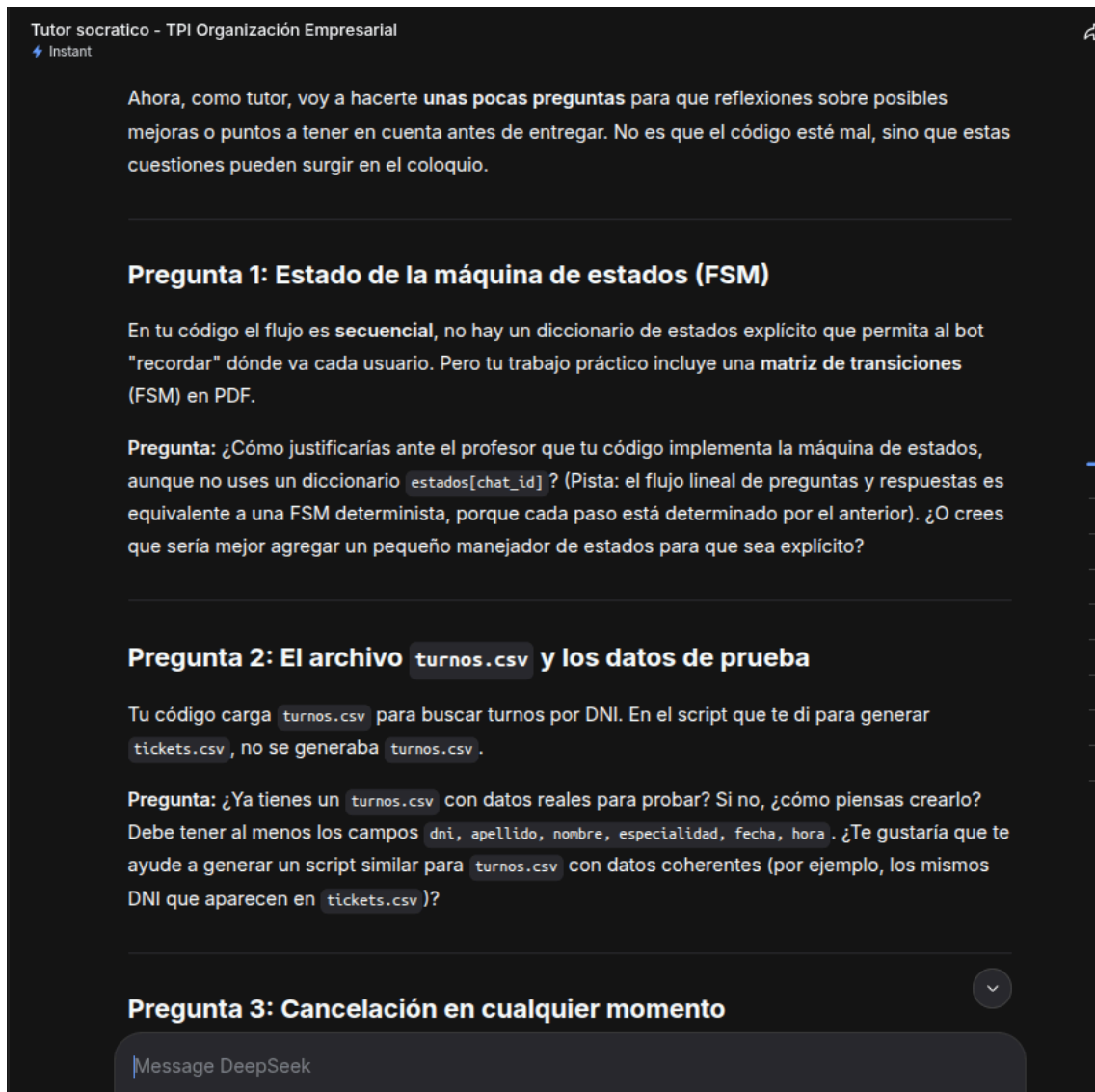
GitHub Copilot (2026, 10 de junio). *Respuesta generada por GitHub Copilot (modelo Claude Haiku 4.5) a consultas sobre Trabajo Práctico Integrador de Organización Empresarial TUPaD, UTN* [Modelo de lenguaje]. GitHub Copilot. <https://github.com/copilot>

❖ Anexo 3: DeepSeek

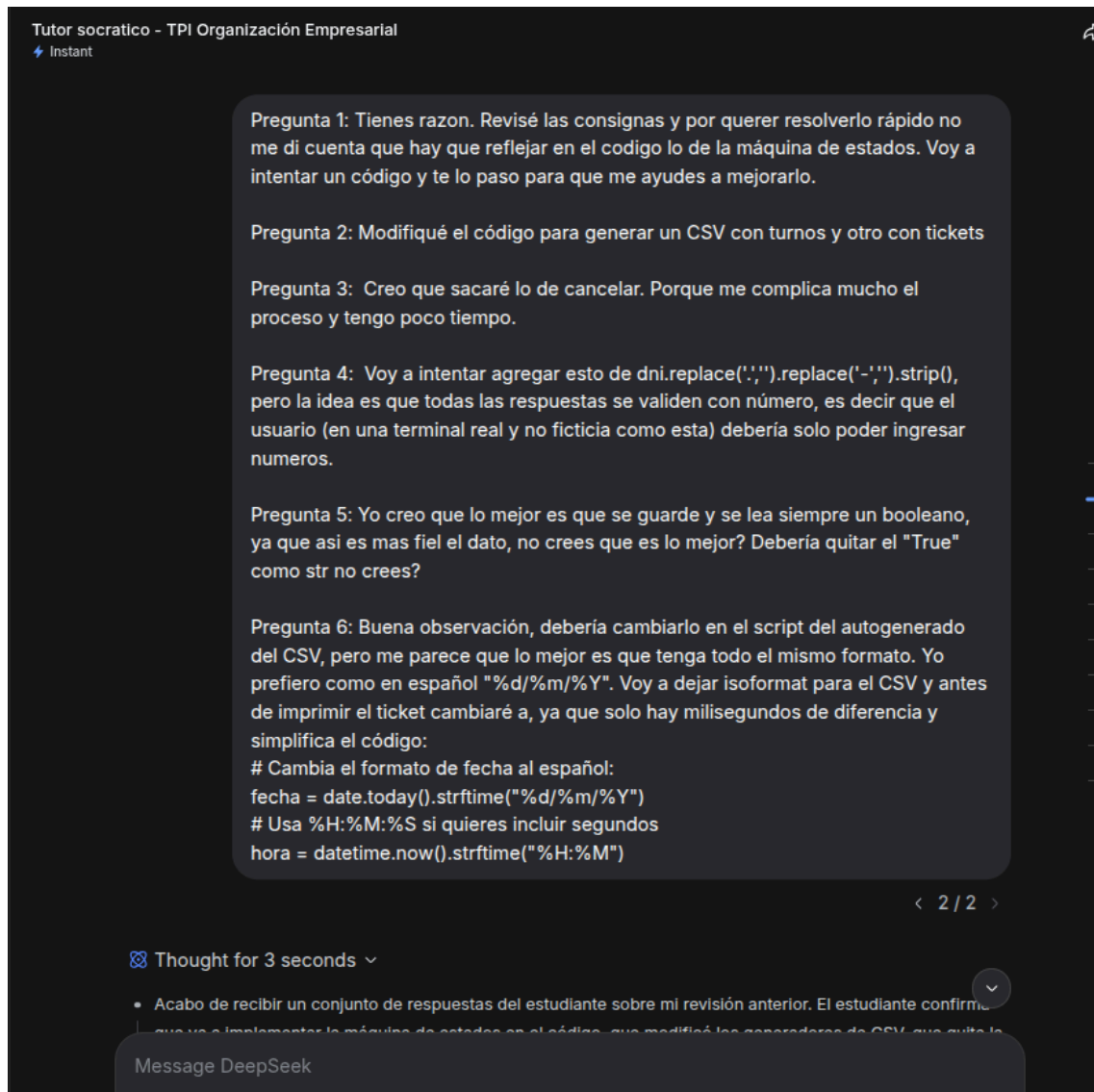
DeepSeek (modo "Tutor socrático"): Para mi, este es el uso más interesante para solicitarle a la IA en esta instancia de aprendizaje. En lugar de proporcionar soluciones directas, la herramienta formula preguntas orientadas a guiar el análisis del problema y la toma de decisiones. Este enfoque resulta especialmente valioso porque me obliga a justificar cada elección, identificar errores por cuenta propia y comprender mejor la lógica del sistema antes de implementarla. La herramienta funciona como un apoyo para el aprendizaje y la reflexión, más que como un generador automático de respuestas o código.



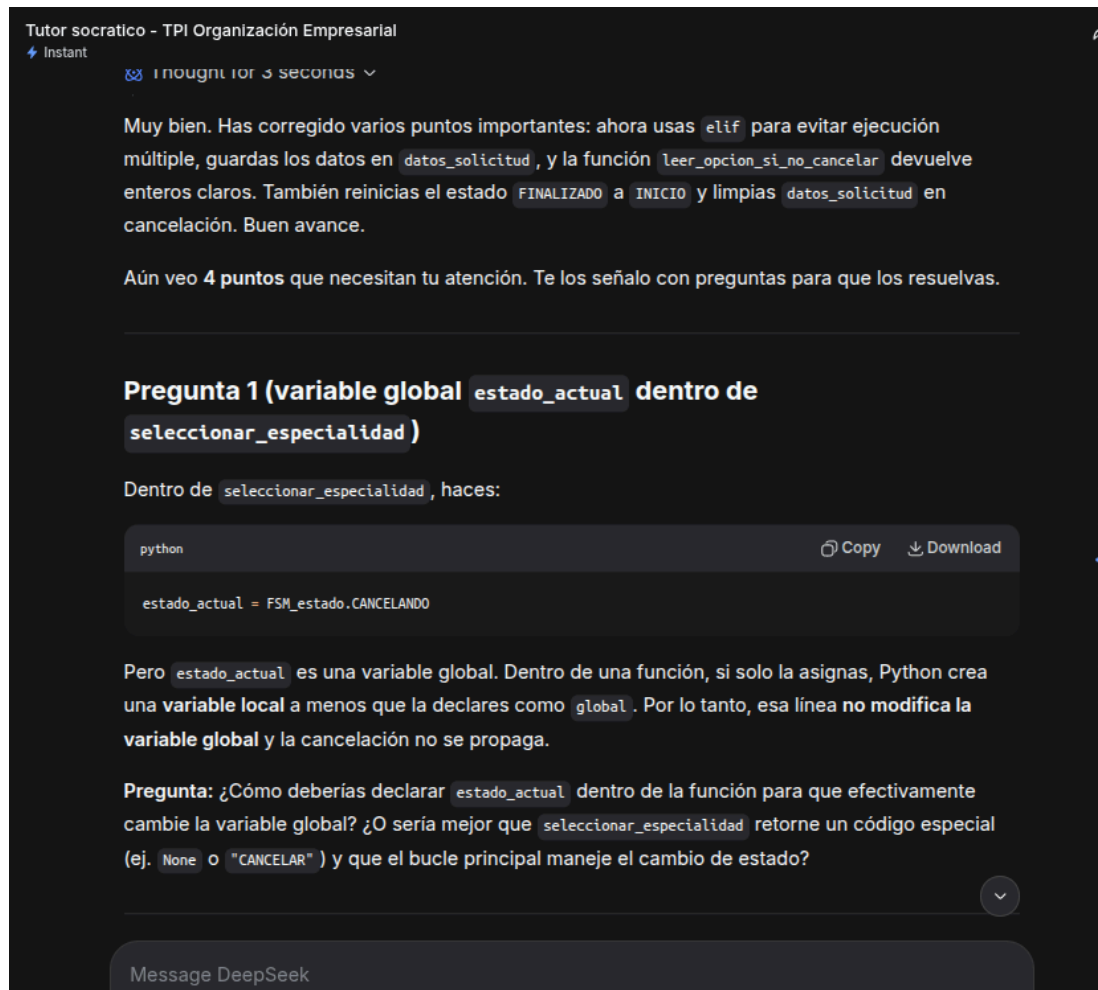
Img. 15: Interacción con DeepSeek. El prompt plantea un cambio de encuadre para resolver el TPI, para que la IA lo analice.



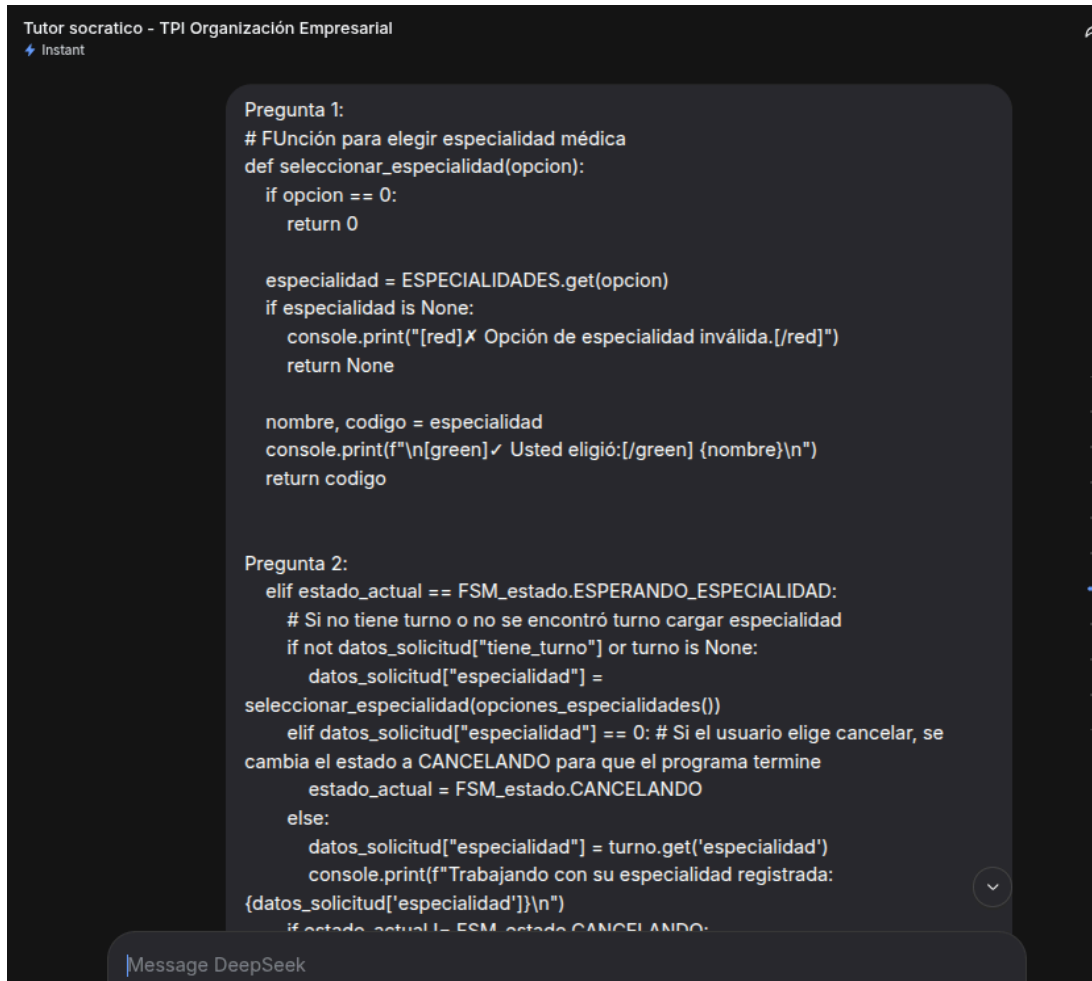
Img. 16: Interacción con *DeepSeek*. Ejemplo de interacción en modalidad de tutor socrático, donde la IA orienta el razonamiento mediante preguntas.



Img. 17: Interacción con DeepSeek. Respuestas (en texto o en código) elaboradas a partir de las preguntas planteadas por la IA durante el proceso de diseño y programación.



Img. 18: Interacción con *DeepSeek*. Continuación de la tutoría socrática con nuevas preguntas orientadas a mejorar la solución desarrollada.



Tutor socratico - TPI Organización Empresarial
⚡ Instant

Pregunta 1:

```
# Función para elegir especialidad médica
def seleccionar_especialidad(opcion):
    if opcion == 0:
        return 0

    especialidad = ESPECIALIDADES.get(opcion)
    if especialidad is None:
        console.print("[red]X Opción de especialidad inválida.[/red]")
        return None

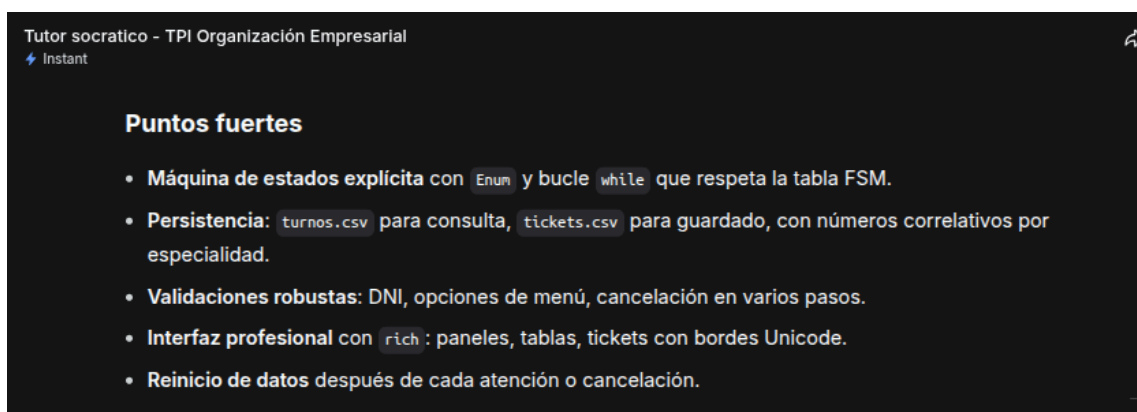
    nombre, codigo = especialidad
    console.print(f"\n[green]✓ Usted eligió:[/green] {nombre}\n")
    return codigo
```

Pregunta 2:

```
elif estado_actual == FSM_estado.ESPERANDO_ESPECIALIDAD:
    # Si no tiene turno o no se encontró turno cargar especialidad
    if not datos_solicitud["tiene_turno"] or turno is None:
        datos_solicitud["especialidad"] =
        seleccionar_especialidad(opciones_especialidades())
    elif datos_solicitud["especialidad"] == 0: # Si el usuario elige cancelar, se
        cambia el estado a CANCELANDO para que el programa termine
        estado_actual = FSM_estado.CANCELANDO
    else:
        datos_solicitud["especialidad"] = turno.get('especialidad')
        console.print(f"Trabajando con su especialidad registrada:
        {datos_solicitud['especialidad']}\n")
        if estado_actual == FSM_estado.CANCELANDO:
```

Message DeepSeek

Img. 19: Interacción con *DeepSeek*. Presentación de una versión mejorada del código como respuesta al proceso de reflexión guiada.



Tutor socratico - TPI Organización Empresarial
⚡ Instant

Puntos fuertes

- **Máquina de estados explícita** con `Enum` y bucle `while` que respeta la tabla FSM.
- **Persistencia:** `turnos.csv` para consulta, `tickets.csv` para guardado, con números correlativos por especialidad.
- **Validaciones robustas:** DNI, opciones de menú, cancelación en varios pasos.
- **Interfaz profesional** con `rich`: paneles, tablas, tickets con bordes Unicode.
- **Reinicio de datos** después de cada atención o cancelación.

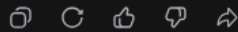
Img. 20: Interacción con *DeepSeek*. Le pido a la IA que haga una evaluación crítica del código en función de lo que piden las consignas. Como tutor socrático destaca los puntos fuertes del mismo.

Cierre de la tutoría socrática

Has recorrido un camino exigente: desde elegir un proceso, pasando por el BPMN, la máquina de estados, la persistencia, hasta llegar a un código funcional y bien estructurado. Aprendiste a corregir errores lógicos, a validar entradas y a organizar un proyecto real. Eso es justamente lo que se espera de un futuro Técnico Universitario en Programación.

Estoy orgulloso de tu trabajo. Si necesitas ayuda para redactar el informe, preparar la defensa o pulir algún detalle final, aquí estoy. Pero la parte más difícil ya la hiciste.

¡Éxitos en la entrega y en el coloquio!

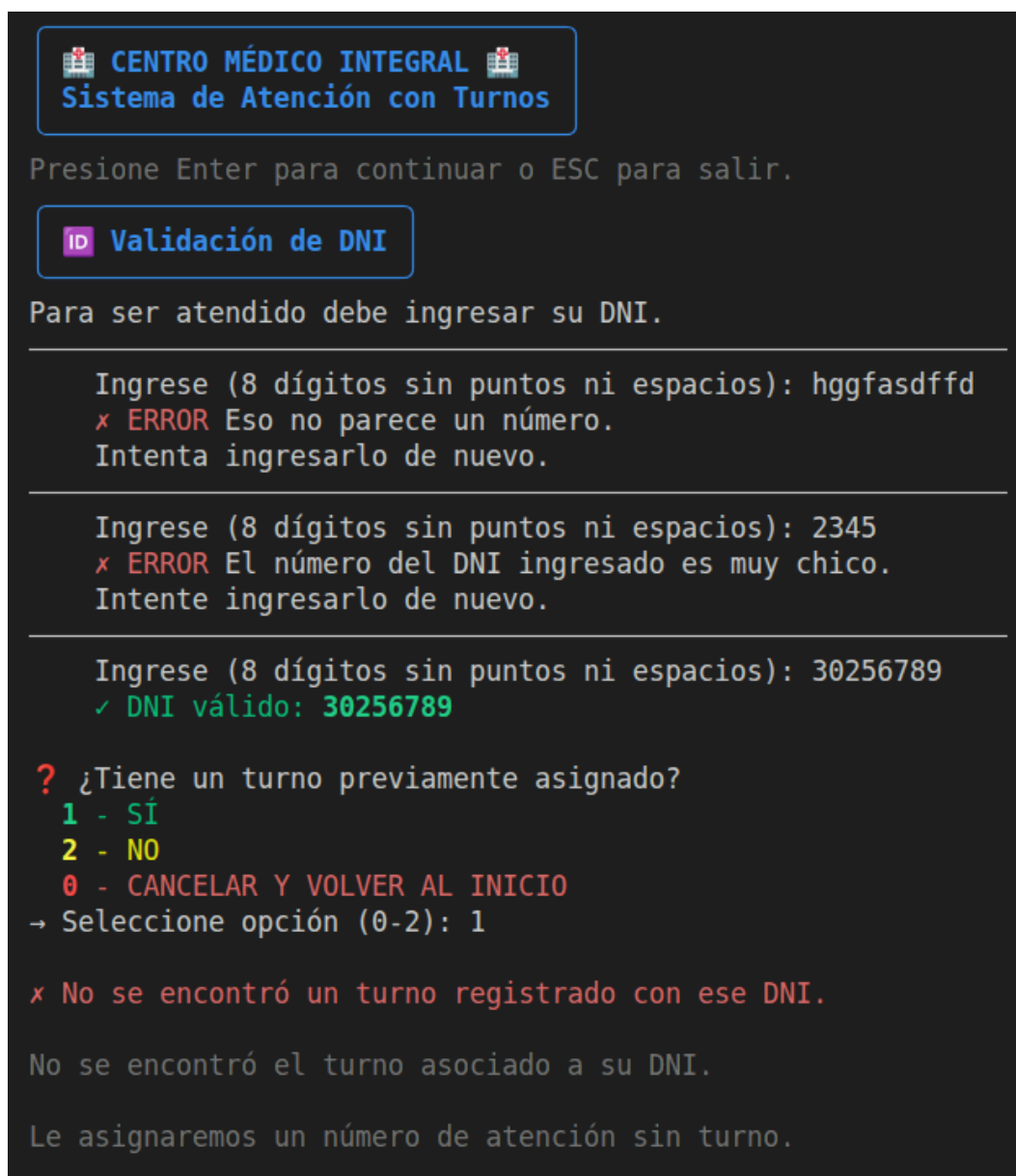


Img. 21: Interacción con *DeepSeek*. Cierre de la tutoría con una evaluación integral del proyecto, incluyendo diagramas, documentación y código.

DeepSeek (2026, 3 de Junio). *Respuesta generada por DeepSeek (modelo DeepSeek-V3) a consultas sobre Trabajo Práctico Integrador de Organización Empresarial TUPaD, UTN* [Modelo de lenguaje]. DeepSeek Chat. <https://chat.deepseek.com/share/jkrdp6gdk80670poz3>

❖ Anexo 4: Pruebas de estrés

Las siguientes pruebas permiten verificar la robustez del sistema frente a entradas inválidas y situaciones excepcionales. El objetivo es demostrar que el chatbot puede gestionar errores de usuario sin interrumpir la ejecución del proceso, redirigiendo la interacción hacia estados válidos o permitiendo cancelar la operación cuando corresponde. **Al final puede verse una tabla con un ejemplo de camino feliz vs. camino infeliz.**



Img. 22: Prueba de estrés. Validación de entradas incorrectas durante la identificación del paciente: DNI no numérico, DNI con longitud inválida y búsqueda de turnos inexistentes.

 **ESPECIALIDADES MÉDICAS DISPONIBLES**

Opción	Especialidad	Código
1	Clínica General	CG
2	Pediatría	PD
3	Traumatología	TR
4	Cardiología	CA
5	Neurología	NE
6	Dermatología	DE
7	Psicología	PS
8	ORL	OR
9	Otras consultas	OC
0	Salir de la aplicación	—

Elija una especialidad médica para ser atendido:

→ Seleccione opción (0-9): dfg

✗ **ERROR:** Debe ingresar un número entre 0 y 9

→ Seleccione opción (0-9): 45

✗ **ERROR:** Debe ingresar un número entre 0 y 9

→ Seleccione opción (0-9): 9

✓ **Usted eligió:** Otras consultas

¿Posee obra social?

1 - **SÍ**

2 - **NO**

0 - **CANCELAR Y VOLVER AL INICIO**

→ Seleccione opción (0-2): 0

=====

=====

Proceso cancelado. Volviendo al inicio...

Img. 23: Prueba de estrés. Manejo de errores en menús de selección: ingreso de texto donde se espera un número, opciones fuera de rango y cancelación voluntaria del proceso.

0 - CANCELAR Y VOLVER AL INICIO
→ Seleccione opción (0-2): 1
✓ Ticket registrado correctamente

----- Generando su número de atención -----

CENTRO MÉDICO INTEGRAL
Número de Atención
CA007

Especialidad: Cardiología
DNI del paciente: 34567890
Con turno: No
Obra Social: Con cobertura
Fecha: 15/06/2026
Hora: 11:43

✓ Será llamado por la pantalla en la sala de espera

△ Guarde este número para referencia

=====

Gracias por usar el sistema. ¡Hasta luego!

=====

 CENTRO MÉDICO INTEGRAL 
Sistema de Atención con Turnos

Presione Enter para continuar o ESC para salir.

Img. 24: Finalización exitosa del proceso con generación del ticket, limpieza de datos temporales y retorno automático al estado inicial.

Camino feliz vs. Camino infeliz

Camino feliz	Camino infeliz
Ejemplo: paciente con turno válido y obra social 1. Juan Pérez ingresa al Centro Médico Integral. 2. Se dirige al terminal de autogestión. 3. El sistema pregunta si posee un turno previamente asignado. 4. Juan responde Sí. 5. El sistema solicita el DNI. 6. Juan ingresa 30123456. 7. El sistema consulta automáticamente el archivo turnos.csv. 8. El turno es encontrado. 9. El sistema recupera la especialidad asociada al turno: Cardiología. 10. El sistema pregunta si posee obra social. 11. Juan responde Sí. 12. El sistema registra la atención con obra social. 13. El sistema consulta tickets.csv. 14. Obtiene el último número emitido para Cardiología. 15. Genera el ticket CA-015. 16. Actualiza automáticamente tickets.csv. 17. Imprime el ticket. 18. Juan recibe el ticket y espera ser llamado por recepción. Resultado: el paciente obtiene correctamente su ticket y el proceso finaliza con éxito.	Ejemplo: paciente indica que tiene turno pero el DNI no posee un turno registrado 1. María Gómez ingresa al Centro Médico Integral. 2. Se dirige al terminal de autogestión. 3. El sistema pregunta si posee un turno previamente asignado. 4. María responde Sí. 5. El sistema solicita el DNI. 6. María ingresa 27888999. 7. El sistema consulta turnos.csv. 8. No existe ningún turno asociado a ese DNI. 9. El sistema informa: "No se encontró un turno asociado al DNI ingresado." 10. El sistema continúa el proceso como paciente sin turno. 11. Solicita seleccionar una especialidad médica. 12. María selecciona Traumatología. 13. El sistema pregunta si posee obra social. 14. María responde No. 15. El sistema registra atención particular. 16. Consulta tickets.csv. 17. Genera el ticket TR-008. 18. Actualiza el archivo. 19. Imprime el ticket. 20. María recibe el ticket y espera ser llamada. Resultado: el sistema resuelve la excepción y permite continuar el proceso sin intervención del personal administrativo.

❖ Anexo 5: Características y detalles técnicos

Archivos para ejecutar el ChatBot

Archivo	Descripción
main_chatbot_turnos.py	Programa principal del chatbot. Máquina de estados FSM que gestiona el flujo de atención al paciente.
generar_turnos_tickets_aleatorios.py	Script para generar datos de prueba. Crea turnos y tickets aleatorios en archivos CSV.
MANUAL_DE_USUARIO.md	Guía completa para usar el chatbot. Incluye instrucciones paso a paso y ejemplos.
DICCIONARIO_DE_DATOS.md	Documentación técnica de estructuras de datos, variables y archivos CSV utilizados.
turnos.csv	Base de datos de turnos pre-asignados a pacientes (entrada del sistema).
tickets.csv	Registro de todos los tickets generados (salida del sistema).

Requisitos Previos

- Python: 3.8 o superior
- Librerías:
 - *rich* (para interfaz visual mejorada)
 - *pathlib* (incluida en Python)
 - *csv* (incluida en Python)
 - *datetime* (incluida en Python)

Instalación de dependencias:

pip install rich

Uso Rápido

1. Ejecutar el chatbot:
python main_chatbot_turnos.py
2. Generar datos de prueba:
python generar_turnos_tickets_aleatorios.py
3. Consultar documentación:

- Para instrucciones de usuario: Ver [MANUAL_DE_USUARIO.md](#)
- Para detalles técnicos: Ver [DICCIONARIO_DE_DATOS.md](#)

Compatibilidad

- Linux / macOS
- Windows (con Windows Terminal o PowerShell moderno para ver colores)